

מבוא לתכנות מערכות – מבחן מועד ב', סתיו תשע"ח

מרצה: ד"ר איתי שרון

משך המבחן: 180 דקות

חומר עזר מותר: 2 דפים המכילים תאור פקודות ב-C ו-bash. כל חומר עזר אחר אסור.

המבחן כולל 5 שאלות, יש לענות על כולן. בשאלות הדורשות נימוק הקפידו לספק אותו.

יש לענות על טופס הבחינה במקומות המיועדים לכך. ניתן להשתמש במחברת העזר אולם יש להגיש רק את טופס הבחינה.

העמוד האחרון במבחן כולל חומר שיכול (אך לא חייב!) לעזור לכם.

בהצלחה!

המכללה האקדמית תל חי, החוג למדעי המחשב

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 1 (15 נקודות)

א. מה עושה הפונקציה הבאה? הניחו שגודל המערך arr הוא n.

```
int foo(unsigned int n, unsigned int arr[])
{
    int i=0, j=0;
    for(; n>0; n--) {
        for(i=2; i<n; i++)
            if(!(n%i))
                break;
        if(i==n)
            *(arr+(j++)) = i;
    }

    return j;
}
```

הפונקציה מוצאת את כל המספרים הראשוניים בין 1 ל-n ומכניסה אותם ל-arr בסדר יורד. הפונקציה מחזירה את

מספר המספרים הראשוניים שנמצאו.

ב. ממשו פונקציה בשם find_max אשר מקבלת מצביע למערך של אלמנטים מטיפוס void*, את גודל המערך ופרמטרים נוספים לבחירתכם ומחזירה את האיבר המקסימלי במערך. ניתן להניח שהמערך מכיל לפחות איבר אחד.

```
void* find_max(void* arr[], int n, bool (*is_bigger)(void* e1, void* e2))
```

```
{
    int i;

    void* emax = arr[0];

    for(i=0; i<n; i++)

        if(is_bigger(arr[i], emax)) {

            emax = arr[i];

        }

    return emax;
}
```

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 2 (30 נקודות)

בשאלה זו אתם מתבקשים לממש באופן חלקי מילון (Dictionary) אשר שומר מילים ואת הפירוש שלהן. המילון ימומש בעזרת עץ חיפוש בינארי אשר ה-header שלו נתון בהמשך. כל מילה והפירוש שלה ישמרו ב-struct אותו תגדירו ושלא יהיה זמין למשתמש. הניחו שכל המילים שמוכנסות למילון הן שונות זו מזו. במימוש של פונקציות אין צורך לבדוק שהקצאות זכרון הצליחו.

```
#ifndef BSTREE_H
#define BSTREE_H
typedef struct BSTree* BSTree;
typedef void* Element;
BSTree BSTCreate(bool (*is_greater)(Element, Element));
void BSTDestroy(BSTree);
BSTree BSTAddNode(BSTree, Element);
BSTree BSTRemoveNode(BSTree, Element);
bool BSTSearchNode(BSTree, Element);
Element* BSTinorder(BSTree);
#endif
```

א. כתבו ממשק (קובץ h) למבנה נתונים Dictionary כמתואר אשר מאפשר את הפעולות הבאות:

- יצירת Dictionary
- הריסת Dictionary
- הוספת מילה חדשה והפירוש שלה למילון.
- הסרת מילה והפירוש שלה מהמילון.
- מציאת פירוש עבור מילה כלשהי במילון. הפונקציה תחזיר את הפירוש, או NULL במידה והמילה לא קיימת במילון.
- החזרת כל המילים שמתחילות ברצף אותיות מסויים אותו מעביר המשתמש. למשל: אם המשתמש העביר "th" תחזיר הפונקציה מערך עם כל המילים שמתחילות ב-th. הפונקציה תקבל כתובת של מערך שהוקצה בצורה דינמית מהמשתמש ותשנה את גודלו בהתאם למספר המילים. הערך המוחזר של הפונקציה יהיה מספר המילים שנמצא.
- החזרת מספר המילים במילון

```
#ifndef DICTIONARY_H
#define DICTIONARY_H

typedef struct Dictionary* Dictionary;

Dictionary CreateDictionary();

void DestroyDictionary(Dictionary dictionary);

bool AddWord(Dictionary dictionary, const char* word, const char* meaning);

bool RemoveWord(Dictionary dictionary, const char* word);

const char* FindMeaning(Dictionary dictionary, const char* word);

int FindWordsBeginWith(Dictionary dictionary, const char* prefix,
                        const char* result[]);

int DictionarySize(Dictionary dictionary);

#endif
```

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. הגדירו את ה-structs הדרושים לצורך מימוש המילון וכן כל פונקציה הנדרשת לצורך השימוש ב-BSTree. ממשו את פונקציית יצירת המילון.

```
#include "Dictionary.h"
```

```
#include "BSTree.h"
```

```
struct WordMeaning {
```

```
    const char* word;
```

```
    const char* meaning;
```

```
};
```

```
bool is_string_greater(Element w1, Element w2) {
```

```
    WordMeaning* pair1 = (WordMeaning*)w1;
```

```
    WordMeaning* pair2 = (WordMeaning*)w2;
```

```
    return (strcmp(w1->word, w2->word) > 0);
```

```
}
```

```
struct Dictionary {
```

```
    BSTree bst;
```

```
};
```

```
Dictionary CreateDictionary() {
```

```
    Dictionary dictionary = (Dictionary)malloc(sizeof(struct Dictionary));
```

```
    dictionary->bst = BSTCreate(is_string_greater);
```

```
    return dictionary;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ג. ממשו את פונקציית הוספת מילה ופירוש למילון. במידה והמילה נמצאת במילון תחזיר הפונקציה false.

```
bool AddWord(Dictionary dictionary, const char* word, const char* meaning) {  
    if (BSTSearch(dictionary->bst, word) != NULL)  
        return false;  
  
    WordMeaning* pair = (WordMeaning*)malloc(sizeof(struct WordMeaning));  
    pair->word = strdup(word);  
    pair->meaning = strdup(meaning);  
    BSTAdd(dictionary->bst, pair);  
    return true;  
}
```


המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 3 (20 נקודות)

א. נתון מערך של מספרים שלמים (int) באורך n כאשר n הוא חזקה של 2 פחות 1 (למשל: 7, 15, 31, 63 וכו). הערכים במערך שונים זה מזה ומכסים את כל הטווח בין 0 ל- $n-1$ חוץ ממספר אחד. לדוגמא: המערך

```
int array[15] = {0, 1, 12, 9, 15, 2, 13, 5, 7, 10, 3, 4, 14, 8, 6};
```

הוא בגודל 15 (2^4-1) והמספר החסר בו הוא 11.

א. ממשו פונקציה אשר מקבלת מערך כמתואר למעלה ומשתנה n אשר מכיל את גודלו ומחזירה את המספר החסר. אין להשתמש בפעולות ביטים בסעיף זה. כמו כן אין להקצות מערכים נוספים ומותר לעבור על המערך הנתון array פעם אחת בלבד.

```
void find missing1(int* array, int n) {
```

```
    int sum = 0, x;
```

```
    for(x=0; x<=n; x++)
```

```
        sum += x;
```

```
    for(x=0; x<n; x--)
```

```
        sum -= array[x];
```

```
    return sum;
```

```
}
```

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. ממשו פונקציה שמוצאת את המספר החסר תוך שימוש בפעולות של ביטים.

```
void find missing2(int* array, int n) {
```

```
    int missing = 0, x;
```

```
    for(x=0; x<=n; x++)
```

```
        sum ^= array[x];
```

```
    return missing;
```

```
}
```

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 4 (20 נקודות)

נתונה מחרוזת s המורכבת מאותיות אנגליות קטנות בלבד. נגדיר פרמוטציה על s ככל מחרוזת אשר מכילה בדיוק את אותן האותיות כמו s אך לא בהכרח באותו הסדר כמו ב- s . המילה עצמה היא גם כן פרמוטציה של עצמה. למשל: עבור המחרוזת "you", הפרמוטציות האפשריות הן "you", "yuo", "uoy", "uyo", "ouy", "oyu".

א. כתבו פונקציה אשר מקבלת שתי מחרוזות $s1$ ו- $s2$ מחזירה true אם $s2$ היא פרמוטציה של $s1$ או false אחרת.

```
bool is_permutation(const char* s1, const char* s2)
```

```
    int difference[26] = {0};
```

```
    int i;
```

```
    while(*s1) {
```

```
        difference[*s1++]++;
```

```
    }
```

```
    while(*s2) {
```

```
        difference[*s2++]--;
```

```
    }
```

```
    for(i=0; i<26; i++) {
```

```
        if(difference[i] != 0)
```

```
            return false;
```

```
    }
```

```
    return true;
```

```
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו פונקציה שמדפיסה למסך את כל הפרמוטציות האפשריות של מילה נתונה.

```
void all_permutations(const int* word, char* perm, int pos, bool* used) {  
    if(pos >= strlen(word)) {  
        printf("%s\n", perm);  
        return;  
    }  
    int i;  
    for(i=0; i<strlen(word); i++) {  
        if(!used[i]) {  
            used[i] = true;  
            perm[pos] = word[i];  
            all_permutations(word, perm, pos+1, used);  
        }  
        knapsack_rec(weights, values, max_weight-weights[j], n, j+1);  
        if(v > value)  
            value = v;  
    }  
    return value;  
}
```

המכללה האקדמית תל חי, החוג למדעי המחשב

שאלה 5 (15 נקודות)

א. כתבו את שורות הקוד הבאות ב-bash:

1. שורת פקודה שמקבלת קובץ בשם foo.txt שבו המילים מופרדות על ידי טאבים ומדפיסה את המילים שנמצאות במקום החמישי בכל שורה בצורה ממויינת לקסיקוגרפית (בהתאם לסידור מילוני).

```
cut -f 5 foo.txt | sort
```

2. שורת פקודה שמוצאת בקובץ foo.txt את כל השורות שבהן מופיע רצף תווים שנראה כמו מספר טלפון סלולרי בישראל: הפורמט הוא 05XYYYYYYY או 05X-YYYYYYY כאשר X ו-Y יכולים להיות כל ספרה בין 0 ל-9. למשל: 050-0505419567, 0505419567.

```
egrep "05[0-9]-?[0-9]{7}" foo.txt
```

3. שורת פקודה (אחת או יותר) שמדפיסה את מספר השורות בכל הקבצים שמסתיימים ב-txt בתיקייה הנוכחית.

```
for x in *.txt ; do
```

```
    wc -l $x
```

```
done
```

אפשר גם

```
wc -l *.txt
```

המכללה האקדמית תל חי, החוג למדעי המחשב

ב. כתבו סקריפט אשר מקבל קובץ שמכיל ציונים של סטודנטים ומדפיס, עבור כל סטודנט, כמה ציונים חסרים לו. הקובץ נתון בפורמט הבא:

- בעמודה הראשונה נמצא השם הפרטי של הסטודנט/ית
- בעמודה השנייה נמצא שם המשפחה
- בעמודות 3-50 נמצאים הציונים. אם הסטודנט סיים את הקורס יופיע ציון בעמודה המתאימה, אחרת יופיע NA.
- העמודות מופרדות ע"י פסיקים.

דוגמא: הנה קובץ ציונים עם 10 ציונים (במקום 50):

```
Itai,Sharon,90,88,56,87,NA,90,98,45,67,NA
John,Dow,NA,70,59,78,67,83,100,22,99,NA
Jane,Dow,78,NA,59,NA,67,83,100,22,99,NA
```

עבור הקובץ הזה ידפיס הסקריפט:

```
Itai Sharon: 2
John Dow: 2
Jane Dow: 3
```

```
#!/bin/bash
```

```
if [[ $# -ne 1 ]] ; then
```

```
    echo -e "\nUsage: $0 <grades-file>\n\n"
```

```
    exit
```

```
fi
```

```
grades=$1
```

```
while read line ; do
```

```
    missing=0
```

```
    for x in `seq 3 30` ; do
```

```
        if [[ `echo $line | cut -d"," -f $x` = "NA" ]] ; then
```

```
            let missing=$missing+1
```

```
        fi
```

```
    done
```

```
    name=`echo $line | cut -f1 -d","`
```

```
    surname=`echo $line | cut -f2 -d","`
```

```
    echo "$name $surname: $missing"
```

```
done < $grades
```


[illegible]

[illegible]

המכללה האקדמית תל חי, החוג למדעי המחשב

bash פקודות

- cat – מדפיסה תוכן של קובץ למסך
- head – מדפיסה את 10 השורות הראשונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- tail – מדפיסה את 10 השורות האחרונות של קובץ למסך (ניתן לשנות את המספר ע"י שימוש ב-n)
- ls – מדפיסה תוכן ספרייה, או רשימה של קבצים אשר מתאימה לתאור שהעביר המשתמש כארגומנט (למשל: ls *.c תדפיס את שמות הקבצים שמסתיימים ב-c. בספרייה הנוכחית)
- sort – ממיינת לקסיקוגרפית קובץ קלט. ניתן למיין מספרית (-n) ולשמור רק עותק אחד של שורות שחוזרות על עצמן (-u)
- wc – מחזירה את מספר השורות (-l), בתים (-c) או מילים (-w) בקובץ
- read – קוראת שורה מה-standard input ומכניסה אותה למשתנה שמועבר ע"י המשתמש כארגומנט
- find – מחזירה רשימה של כל הקבצים אשר מתאימים לתאור המשתמש (ארגומנט -name) בספרייה מסויימת ואלו שנמצאות תחתיה (ארגומנט ראשון לפקודה)
- grep – מחזירה שורות אשר מכילות string מסויים. ניתן להשתמש ב-regular expressions (-e), ולהחזיר שורות שאינן מכילות את ה-string אותו מחפשים (-v)
- egrep – זהה ל-grep -e
- cut – מחזירה שדות מסויימים בכל שורה (-f) בהתאם ל-delimiter (-d). ניתן גם להחזיר בתים מסויימים בכל שורה (-b)
- sed – עורכת טקסט הנתון כקלט. הפעולה s תשנה שורת טקסט.

פעולות על ביטים

- bitwise AND - &
- bitwise OR - |
- bitwise XOR - ^
- right shift - >>
- left shift - <<
- One's complement - ~

פונקציות ב-C

- int putchar(int character, FILE* fp) – כותבת את character ל-fp.
- int fgetc(FILE* fp) – קוראת תו אחד מתוך fp.
- scanf(const char* format,...) – קוראת מידע מתוך stdin בהתאם ל-format, מאחסנת את המידע בכתובות המשתנים שמסופקים. fscanf(FILE* stream, const char* format,...) – אותו דבר כאשר הקלט נקרא מתוך stream.
- sscanf(const char* s, const char* format,...) – אותו הדבר כאשר הקלט נקרא מתוך s.
- int getline(char** lineptr, size_t* n, FILE* stream) – קוראת שורה לתוך lineptr, מקצה זכרון עבור lineptr אם אין מספיק (גודל ה-buffer הנוכחי מצוי ב-n). מחזירה את מס' התווים שקראה או -1 אם לא הצליחה לקרוא.
- printf(const char* format,...) – כתיבת מידע ערוך ל-stdout בהתאם ל-format והמשתנים שמסופקים.
- fprintf(FILE* stream, const char* format,...) – אותו דבר כאשר הכתיבה נעשית לתוך stream.
- sprintf(const char* s, const char* format,...) – אותו הדבר כאשר הכתיבה נעשית לתוך s.
- void* malloc(size_t size) – מקצה זכרון בגודל size
- void* calloc(size_t num, size_t size) – מקצה num יחידות בגודל size כל אחת ומאפסת את כל הזכרון שהוקצה.
- void* realloc(void* ptr, size_t size) – מקצה size בתים, מוסיפה/מורידה מכמות הזכרון ש-ptr מצביעה עליו (צריך להיות מוקצה דינמית גם כן), עשויה לשנות את הזכרון המוקצה ואז תעתיק את התוכן של ptr לשטח החדש. מחזירה את כתובת הזכרון שהוקצה.
- const char* strstr(const char* str1, const char* str2) – מחזירה את המקום שבו מופיעה str2 ב-str1 (או NULL אם היא לא מופיעה)
- int strcmp(const char* str1, const char* str2) – משווה את str1 ל-str2, מחזירה 0 אם הן זהות, ערך קטן מ-0 אם str1 קטן לקסיקוגרפית מ-str2 או ערך גדול מ-0 אחרת.

המכללה האקדמית תל חי, החוג למדעי המחשב

- `size_t strlen(const char* str)` – מחזירה אורך של מחרוזת.
- `int fseek(FILE *fptr, long offset, int origin)` – מזיזה את `fptr` ל-`offset` בתיים ביחס ל-`origin`. המשתנה `origin` יכול להיות
 - `SEEK_SET` – תחילת הקובץ
 - `SEEK_CUR` – מיקום נוכחי
 - `SEEK_END` – סוף הקובץ

הפונקציה תחזיר 0 בהצלחה או ערך שונה מ-0 אחרת.

Regular expressions

.	Match any character
*	Match 0 or more time the preceding character
+	Match 1 or more time the preceding character
?	Match 0 or 1 times the preceding character
{n}	Match exactly n of the preceding item
{n, m}	Match between n and m of the preceding item
{n,}	Match n or more of the preceding item
^	Beginning of line
\$	End of line
[]	Match a range of characters
[^]	Match a character which is not contained in the range
	Or – match either the pattern before or after the
()	Group together parts of the search expression